CHAPTER 5 — SOFTWARE DESIGN

Descriptive & Definitional Questions

Section 1: Foundations, Objectives & Modularity

Q5.D01. What is software design, and what specific role does it perform within the Software Development Life Cycle (SDLC)?

Software design is the first stage in the development phase of the software life cycle, representing the creative engineering process that transforms the requirements specification into a blueprint for construction. In this process, the problem description formulated during analysis is systematically mapped into an operational software architecture.

Role within the SDLC:

1. **Bridge Between What and How:** Requirements analysis describes *what* the system must do from the external user perspective; software design specifies *how* the system will achieve those objectives structurally, architecturally, and procedurally.
2. **Baseline for Implementation and Verification:** Design produces technical specifications (architecture, interfaces, data structures, and algorithms) that developers use to write code and testers use to formulate verification and unit/integration test suites.
3. **Managing Program Size and Complexity:** For large programs, the cognitive gap between an external specification and executable code is vast. Structured design decomposes complex systems into manageable, interacting modules, making development, tracking, and future modifications feasible.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 203–204; Companion Slides 2–4. Exercise 5.1

Q5.D02. Contrast the scope, audience, activities, and deliverables of Conceptual Design versus Technical Design.

Software design is partitioned into two distinct collaborative phases:

1. **Conceptual Design (External / Customer-Oriented Design):**
2. **Primary Audience:** System users, business clients, and domain stakeholders.
3. **Core Purpose:** To establish a shared, non-technical understanding of how the system will interact with its external environment and fulfill functional requirements.
4. **Key Activities:** Defining user workflows, external interface mockups, screen layouts, input/output data formats, and business rules without referencing software implementation specifics.
5. **Primary Output:** Conceptual Design Document / Functional Specification, expressed in clear business and domain terminology.
6. **Technical Design (Internal / Developer-Oriented Design):**
7. **Primary Audience:** Software architects, lead programmers, database administrators, and systems integrators.
8. **Core Purpose:** To define the internal engineering structures, computational mechanisms, and hardware/software interfaces required to construct the software.
9. **Key Activities:** Module decomposition, control hierarchies, interface signatures, relational/object database schemas, data structure selections, coupling/cohesion optimization, and detailed algorithmic logic.
10. **Primary Output:** Software Design Description (SDD) conforming to formal standards such as IEEE Std 1016.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 204–205; Companion Slides 4–5, Fig. 2. Exercise 5.1

Q5.D03. What are the primary objectives of software design, and why is the design phase important?

The four design objectives are:

1. **Correct and complete** — satisfy the stated requirements.
2. **Understandable** — make the design clear to its readers.
3. **At the right level** — provide detail appropriate to the design stage and its users.
4. **Maintainable, and facilitate maintenance** — make later changes practical.

Vitruvius's three qualities are **durability, utility, and charm**. Design matters because it translates requirements into a construction plan and affects the quality and maintainability of the resulting software.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §5.1.2, printed pp. 205–206; Exercise 5.2.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - احفظ الأهداف الأربعة بلفظها؛ وخصائص Vitruvius هي durability, utility, charm، مو structural coherence.

Q5.D04. Clearly distinguish between Software Design and Coding. Why is jumping directly to coding considered a detrimental engineering anti-pattern?

Software Design vs. Coding:

Dimension	Software Design	Coding (Implementation)
Primary Concern	Structural architecture, modular decomposition, component interactions, interface contracts, and data abstractions.	Syntactic translation of design algorithms and data structures into an executable programming language.
Cognitive Focus	Global perspective: system-level trade-offs, modular independence, quality attributes, and maintainability.	Local perspective: statement semantics, language idioms, variable lifetimes, compilation, and syntax compliance.
Level of Abstraction	High to medium abstraction (architectural blueprints, class models, structure charts, schemas).	Low abstraction (concrete statements, library calls, memory management, pointers/references).
Change Cost Impact	Architectural changes are simulated and adjusted on paper or models at negligible cost.	Structural rework requires rewriting, recompiling, re-testing, and debugging hundreds of lines of code.

The Anti-Pattern of Immediate Coding ("Code-and-Fix"):

Commencing implementation without an explicit, validated design is known as the "Code-and-Fix" anti-pattern. Developers who jump straight to coding focus prematurely on localized syntax and immediate execution, losing sight of overall modular coupling, cohesion, and scalability. This leads directly to tightly coupled, monolithic "spaghetti code", undetected requirements omissions, impossible regression testing, and massive technical debt that quickly paralyzes future development.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 203–204; Companion Slide 3. Exercise 5.3

Q5.D05. Define Modularity. State and explain Bertrand Meyer's five modularity criteria (Decomposability, Composability, Understandability, Continuity, and Protection).

Definition of Modularity:

Modularity is the software architectural attribute whereby a system is partitioned into discrete, logically cohesive, and loosely coupled units (modules) that interact through cleanly defined interfaces. Modularity embodies the foundational principle of "divide and conquer", allowing complex systems to be designed, understood, implemented, and maintained by decomposing them into intellectually manageable pieces.

Bertrand Meyer's Five Modularity Criteria:

To ensure that modular decomposition achieves robust software engineering quality, Bertrand Meyer established five formal criteria:

1. **Modular Decomposability:**

2. *Criterion:* A design method satisfies modular decomposability if it provides a systematic mechanism for breaking a complex problem into a hierarchy of smaller, simpler sub-problems that can each be resolved independently.
3. *Rationale:* Decomposability enables division of labor across engineering teams and reduces cognitive load during development. Top-down functional decomposition is a canonical example.
4. **Modular Composability:**
5. *Criterion:* A design method satisfies modular composability if it enables modules designed for one system to be freely recombined with existing or newly created modules to construct completely different systems.
6. *Rationale:* Composability maximizes software reusability. It requires modules to be self-contained and free of hardcoded assumptions regarding their operational environment (e.g., standard math libraries or Unix pipelines).
7. **Modular Understandability:**
8. *Criterion:* A design method satisfies modular understandability if an engineer can inspect and comprehend an individual module's behavior without having to read or understand the internal implementations of other modules in the system.
9. *Rationale:* If comprehending a module requires tracing the execution of ten other modules, the system lacks modular understandability, making maintenance error-prone and costly.
10. **Modular Continuity:**
11. *Criterion:* A design method satisfies modular continuity if a small modification to the system requirements specification results in changes to only one module or a minimal, localized cluster of modules, without rippling across the architectural boundary.
12. *Rationale:* High continuity protects the architecture against catastrophic regression. If adding a minor business rule requires updating every module in the project, the design fails modular continuity.
13. **Modular Protection:**
14. *Criterion:* A design method satisfies modular protection if an exceptional condition, run-time error, or fault occurring inside a module remains localized within that module, or at worst propagates to strictly adjacent calling modules, without crashing the entire system.
15. *Rationale:* Robust architectures prevent fault propagation through defensive programming, input validation, and structured exception handling at module boundaries.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 206–207; Companion Slides 14–15; Pressman (5th ed.), §13.4.3, printed p. 345 / PDF p. 373. Exercise 5.4

Section 2: Coupling and Cohesion Taxonomies

Q5.D06. Define Module Coupling. Enumerate and define all six primary levels of coupling from most desirable (loosest) to least desirable (tightest).

Definition of Module Coupling:

Module coupling is the qualitative measure of the degree of interdependence and structural connectivity between two software modules. It indicates how closely bound two modules are, and the ease with which one module can be understood, modified, or replaced without destabilizing the other. Software engineering strives for *loose coupling*.

The Six Levels of Coupling (Ranked from Best to Worst):

1. **Data Coupling (Most Desirable / Lowest Coupling):**
2. *Definition:* Communication occurs strictly through passed argument parameters that are elemental, atomic data types (e.g., primitive integers, booleans, or floats).
3. *Characteristic:* Neither module exposes internal representation; data flows cleanly across the parameter list.
4. **Stamp Coupling:**
5. *Definition:* Modules communicate by passing a composite, structured data type (e.g., a record, struct, or class object), but the called module uses only a subset of the fields within that structure.
6. *Characteristic:* Creates unnecessary data exposure; altering unused fields in the record may require recompiling or re-verifying the called module.
7. **Control Coupling:**
8. *Definition:* One module passes an explicit control flag, opcode, or switch variable to another module with the explicit intent of directing its internal logic and procedural execution path.
9. *Characteristic:* The calling module must possess intimate knowledge of the callee's internal algorithmic branching.
10. **External Coupling:**
11. *Definition:* Modules communicate by binding to an externally imposed communication environment, format, or hardware interface (e.g., standard OS devices, communication protocols, or vendor file formats).
12. *Characteristic:* Changes to the external protocol ripple into every module bound to that interface.
13. **Common Coupling:**
14. *Definition:* Multiple modules access, read, and write to a shared, global data environment (e.g., global variables, shared memory pools, or unbounded public static records).
15. *Characteristic:* A bug in one module corrupts the shared memory, creating side effects that manifest unpredictably across distant modules.
16. **Content Coupling (Worst / Highest Coupling):**
17. *Definition:* One module directly references, modifies, or branches into the internal implementation code, local variables, or execution pointers of another module.

18. *Characteristic:* Utter destruction of modular boundaries; neither module can be tested, refactored, or understood in isolation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 207–210; Companion Slides 15–24. Exercise 5.5

Q5.D07. Define Module Cohesion. Enumerate and define all seven levels of cohesion from least desirable (weakest) to most desirable (strongest).

Definition of Module Cohesion:

Module cohesion is the qualitative measure of the internal functional strength, singularity of purpose, and degree to which all elements within a single module belong together. A module has high cohesion if its internal instructions and data structures cooperate exclusively to execute a single, focused task. Software engineering strives for *high cohesion*.

The Seven Levels of Cohesion (Ranked from Worst to Best):

1. **Coincidental Cohesion (Weakest / Least Desirable):**
2. *Definition:* Instructions are collected into a module arbitrarily with no meaningful functional, sequential, or temporal relationship.
3. *Characteristic:* Often created when disparate lines of repeated code are lazily lumped into a general "utilities" routine.
4. **Logical Cohesion:**
5. *Definition:* Elements perform a set of logically categorized functions (e.g., all output routines, all mathematical operations), and a control flag passed at runtime dictates which specific function executes.
6. *Characteristic:* Code for distinct tasks is interwoven in a large switch/if statement, making changes to one function risk corrupting others.
7. **Temporal Cohesion:**
8. *Definition:* Tasks are grouped together inside a module simply because they must all be executed within the same window of time during system execution.
9. *Characteristic:* Classical examples include initialization routines (`init_all_devices()`) or shutdown procedures (`cleanup_system()`).
10. **Procedural Cohesion:**
11. *Definition:* Elements are executed in a specified procedural sequence (an algorithmic order), but they do not operate on the same data.
12. *Characteristic:* The sequence resembles a flowchart segment (e.g., "read record, compute taxes, print receipt") grouped into a single unit.
13. **Communicational Cohesion:**
14. *Definition:* Module elements execute different operations, but they all operate upon the exact same input data structure or produce the exact same output data.
15. *Characteristic:* Example: A module that takes a student record and calculates GPA, determines honors eligibility, and prints a transcript.
16. **Sequential Cohesion:**
17. *Definition:* The output produced by one operational step serves as the direct input to the immediately following operational step, forming a computational assembly line.

18. *Characteristic:* Strong cohesion, but slightly less focused than functional cohesion because it contains multiple distinct transformations.
19. **Functional Cohesion (Strongest / Most Desirable):**
20. *Definition:* Every instruction and element inside the module cooperates exclusively to perform a single, cleanly defined functional task.
21. *Characteristic:* Canonical examples include pure mathematical calculations (calculate_cosine( )), cryptographic hashing (compute_sha256( )), or well-bounded domain queries.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 210–213; Companion Slides 25–30. Exercise 5.6

Section 3: Design Notations, Strategies & Function-Oriented Design

Q5.D08. Why do software engineers employ different design notations (e.g., DFDs, Structure Charts, UML diagrams) rather than a single unified representation?

Reasons for Employing Multiple Design Notations:

No single diagrammatic notation can simultaneously capture all dimensions of a software architecture without suffering catastrophic visual clutter and cognitive overload. Complex software systems possess multi-dimensional properties that require specialized visual notations tailored to specific analytical views:

1. **Data-Flow Perspective vs. Control Hierarchy:**
2. **Data Flow Diagrams (DFDs):** Emphasize data movement, transformations, sources, sinks, and stores throughout the system, intentionally omitting control logic, loop counters, and sequence timing.
3. **Structure Charts:** Model procedural control hierarchies, showing module invocation, parameter passing, and control flags, which DFDs cannot represent.
4. **Static Architecture vs. Dynamic Behavior:**
5. **Static Models (Class Diagrams, Component Diagrams):** Depict structural relationships, class inheritance, dependencies, and packaging that remain invariant during execution.
6. **Dynamic Models (Sequence Diagrams, State Machine Diagrams, Activity Diagrams):** Trace temporal event ordering, message lifetimes, and state transitions across operational execution paths.
7. **Diverse Stakeholder Perspectives:**
8. Non-technical business stakeholders require high-level Use Case Diagrams and Context DFDs to validate business boundaries.
9. Software architects require Component and Deployment Diagrams to evaluate physical node allocation and communication bottlenecks.
10. Programmers require detailed Structure Charts and Sequence Diagrams to write algorithmic code and handle edge cases.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–221; Companion Slides 41–46. Exercise 5.14

Q5.D09. In Function-Oriented Design, what are the Functional Procedure Layers, and what information belongs at each level?

The four levels specify increasing detail for an individual function or procedure:

1. **Level 0:** Function name, its purpose, its place in the system, the routines that call it, and authorship/date.
2. **Level 1:** Parameters, global variables, routines called, side effects, and input/output assertions.
3. **Level 2:** Local data structures, timing constraints, exception conditions and responses, and limitations.
4. **Level 3:** The function body, expressed using a structured chart, pseudocode, decision table, flowchart, or another suitable notation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 220–221; Companion Slides 47–49; Exercise 5.15.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - المقصود مستويات تفصيل مواصفة الإجراء من Level 0 إلى Level 3، مو طبقات معمارية مثل Control/Transform/Utility.

Section 4: Object-Oriented Design Concepts & Standards

Q5.D10. Define the fundamental object-oriented concepts: object, class, message passing, abstraction, inheritance, and polymorphism.

Object: An entity with identity, state, and behavior. **Class:** A set of objects sharing common characteristics; it describes their common structure and behavior and serves as a blueprint for objects.

Message passing is communication in which an object requests an operation from another object. **Abstraction** focuses on relevant properties while leaving other details aside. **Inheritance** defines a more specific class in relation to a more general one. **Polymorphism** allows the same operation or message to have different implementations for different object types.

A class and an abstract data type are related ideas, but they are not interchangeable definitions: an ADT is specified through its operations while its representation is hidden.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 226–230; Sommerville (9th ed.), §13.4, PDF p. 356; Exercise 5.16.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - لا تكتب أن class هو نفسه ADT؛ تعريف الصنف عند أغروال يجمع الكائنات ذات الخصائص/السلوك المشترك.

Q5.D11. What improvements can Object-Oriented Design offer over Function-Oriented Design?

Function-Oriented Design organizes a system around functions and their decomposition. Object-Oriented Design focuses on the data manipulated by the program and models problem-domain things as objects with state and behavior. Grouping common characteristics into classes, using abstraction, and organizing related classes through inheritance can help represent the problem at suitable levels. Encapsulation can hide implementation details behind an interface, helping localize some changes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–220, 225–230; Exercise 5.19.

Q5.D12. Using this chapter's object-oriented concepts, state practical guidelines for producing good-quality or reusable classes.

1. **Model a meaningful concept:** Use a class for objects that share common structure and behavior.
2. **Choose a suitable abstraction level:** Keep essential characteristics visible and leave irrelevant details aside.
3. **Keep state and operations coherent:** Describe the class's attributes and the operations that examine or affect its state.
4. **Hide implementation details:** Expose an interface for using the class while keeping internal implementation details encapsulated.
5. **Use inheritance for genuine generalization:** Place shared characteristics in a suitable general class and specialize below it.

These are practical guidelines synthesized from the chapter's class, abstraction, encapsulation, and hierarchy concepts.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 226–231; Exercise 5.20.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - هذا جواب تركيبي من مفاهيم الفصل؛ لا تنسب إليه SOLID أو Law of Demeter أو قواعد access modifiers غير الموجودة في الصفحات المذكورة.

Q5.D13. Outline the process used to analyze and design an object-oriented system.

The process follows these activities:

1. Create a **use-case model**: identify actors, write use cases, and draw the use-case diagram.
2. Create an **activity diagram**, if required.
3. Create **interaction diagrams**: identify objects for each use case and prepare sequence/collaboration diagrams for each use case.
4. Create a **class diagram**.

5. Create **state-chart and object diagrams**, if required.
6. Create **component and deployment diagrams**.
7. Prepare the **design documents**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §5.6.2, printed pp. 230–233; Companion Slides 72–75; Exercise 5.21.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - انتبه إلى عبارتي "if required" في النشاط ومخططات الحالة/الكائن؛ لا تحوّل الخطوات الشرطية إلى إلزامية.

Q5.D14. What documents should be produced on completion of the design phase, and what information does the SDD organize?

The design phase should leave a Software Design Description (SDD) and the design views/artifacts needed to communicate the system's structure, relationships, interfaces, and internal details. Depending on the design, these may include hierarchical decomposition diagrams, structure charts, data-flow or transaction diagrams, interface descriptions, flowcharts, and program-design-language descriptions.

The course material lists these **10 design-entity attributes**: Identification, Type, Purpose, Function, Subordinates, Dependencies, Interface, Resources, Processing, and Data. Its four design views are **Decomposition**, **Dependency**, **Interface**, and **Detail Description**. The **Function** attribute appears in the Decomposition view.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 221–225; Companion Slides 53–57; Exercise 5.23.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني: - اذكر SDD والرسوم/الأوصاف اللازمة، ثم ميّز بين خصائص كيان التصميم والـfour design views. لا تضف تسليمات ثابتة مثل integration-test plan من دون سند.

Section 5: Doctor Seminar Synthesis (Abstraction, Refinement, Hiding & Reviews)

Q5.D15. Explain the three fundamental forms of abstraction in software engineering: Procedural Abstraction, Data Abstraction, and Control Abstraction.

The Three Fundamental Forms of Abstraction:

Abstraction is the intellectual mechanism that enables engineers to manage complexity by concentrating on essential characteristics while suppressing localized details. In software architecture, abstraction manifests in three primary forms:

1. **Procedural Abstraction:**
2. **Definition:** A named sequence of instructions that performs a specific, well-defined function and can be referenced as a single operational unit.
3. **Mechanism:** The caller understands *what* the procedure accomplishes through its name and parameter interface (e.g., sort_records(array) or open_network_socket()), without knowing whether the sorting algorithm is quicksort, mergesort, or heapsort.
4. **Data Abstraction:**
5. **Definition:** A named collection of data elements and the set of valid operations that can be performed upon that data, while completely concealing internal representation and memory layout.
6. **Mechanism:** Often realized as Abstract Data Types (ADTs) or classes (e.g., a Queue or SymbolTable). The user interacts strictly through abstract operations (enqueue, dequeue), remaining isolated from whether the underlying implementation uses a dynamic circular array or a singly linked list.
7. **Control Abstraction:**
8. **Definition:** High-level mechanisms that coordinate the sequencing, scheduling, synchronization, and branching of execution without detailing low-level machine registers or branch instructions.
9. **Mechanism:** Examples include structured loop constructs (foreach), concurrency primitives (semaphores, mutexes, monitors), event-driven publish-subscribe dispatchers, and asynchronous pipeline handlers.

Source: Pressman (5th ed.), §13.4.1, printed pp. 342–343 / PDF pp. 370–371; K.K. Aggarwal pp. 206, 226.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- أشكال التجريد الثلاثة التي شدد عليها الدكتور في ملاحظات السيمنار:
- تجريد إجرائي (Procedural Abstraction): إخفاء خوارزمية التنفيذ داخل إجراء ذي اسم وواجهة واضحة.
- تجريد بياني (Data Abstraction): إخفاء طريقة تخزين البيانات وتوفير عمليات وصول مقننة (ADTs/Classes).
- تجريد التحكم (Control Abstraction): إخفاء آليات التزامن وتوجيه التنفيذ خلف هياكل تحكم عليا.

Q5.D16. Explain the principles of Stepwise Refinement (Niklaus Wirth) and Information Hiding (David Parnas), and show how they guide effective module decomposition.

Principles of Stepwise Refinement and Information Hiding:

1. Stepwise Refinement (Niklaus Wirth):

- **Definition:** A top-down architectural strategy wherein a high-level, abstract statement of a program's function is systematically decomposed into progressively lower-level, more detailed procedural and data specifications through successive iterations.
- **Operational Flow:** At each iteration, abstract functional statements are replaced by a combination of simpler sub-functions and concrete data structures, continuing downward until the statements correspond directly to programming language primitives.
- **Decomposition Value:** Prevents premature fixation on low-level coding details, preserving architectural clarity from initial concept down to final implementation.

2. Information Hiding (David Parnas):

- **Definition:** The fundamental design heuristic stating that each module should encapsulate and conceal a specific design secret (such as a complex algorithm, hardware interface, or internal data representation) behind a stable, public interface.
- **Operational Flow:** Modules communicate strictly through published interface contracts. Other system components cannot access or depend upon the internal representation.
- **Decomposition Value:** Information hiding maximizes modular independence and ensures high modular continuity: when an internal secret changes (e.g., migrating from an in-memory B-Tree to an indexed SQL table), only the encapsulating module is modified; calling modules remain unaffected.

Source: Pressman (5th ed.), §13.4.2 & §13.4.4, printed pp. 343–345, 352 / PDF pp. 371–373, 380; David L. Parnas (1972).

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- التحسين التدريجي (Stepwise Refinement): الانتقال المنظم من التجريد العالي إلى كود تفصيلي خطوة بخطوة.
- إخفاء المعلومات (Information Hiding - Parnas): كل وحدة برمجية تخفي سراً تصميمياً واحداً خلف واجهة مستقرة، وهو الأساس المتين لتقليل الاقتران وضمان عدم انتشار آثار التعديلات المستقبلية.

Q5.D17. What is the purpose of a software design review, who commonly participates, and what is reviewed?

A software design review examines a proposed design to identify problems before implementation. A review team commonly brings together people representing design, implementation, testing, and maintenance perspectives; typical participants include designers, coders, testers, analysts, and maintainers. The team examines the design's consistency, interfaces, and suitability for implementation and later maintenance.

Source: Rajib Mall (4th ed.), §6.7, printed p. 350; professor-seminar coverage.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني: - هؤلاء مشاركون معتادون بحسب منظور المراجعة؛ لا تحفظهم كستة أدوار إلزامية. حذفت نسبة 10–100× لأنها غير مسندة هنا.

Topic-to-Question Map

Topic Area	Questions Covering Item	Primary Course Sources	Key Concepts Covered
1. Software Design Fundamentals	Q5.D01, Q5.D02, Q5.D03, Q5.D04	K.K. Aggarwal pp. 203–206; Slides 2–10	Design definition, SDLC role, conceptual vs technical design, design objectives, design vs coding.
2. Modularity & Meyer's Criteria	Q5.D05	Pressman §13.4.3; K.K. Aggarwal pp. 206–207	Modularity definition, Meyer's 5 criteria (decomposability, composability, understandability, continuity, protection).
3. Module Coupling Taxonomy	Q5.D06	K.K. Aggarwal pp. 207–210; Slides 15–24	Coupling definition, 6 coupling levels (data, stamp, control, external, common, content).
4. Module Cohesion Taxonomy	Q5.D07	K.K. Aggarwal pp. 210–213; Slides 25–30	Cohesion definition, 7 cohesion levels (coincidental, logical, temporal, procedural, communicational, sequential, functional).
5. Design Notations & FOD	Q5.D08, Q5.D09	K.K. Aggarwal pp. 216–221; Slides 36–49	Multi-view notations, DFD vs structure chart, functional procedure layers, structured design techniques.
6. Object-Oriented Design (OOD)	Q5.D10, Q5.D11, Q5.D12, Q5.D13	K.K. Aggarwal pp. 225–233; Slides 58–75	OOD core concepts, OOD vs FOD advantages, high-quality class heuristics, OOAD 6-step process.
7. IEEE SDD (Std 1016-1998)	Q5.D14	K.K. Aggarwal pp. 221–225; Slides 50–57	IEEE Std 1016 scope, definitions, purpose, 10 entity attributes, 4 design views, post-design documents.
8. Seminar Gaps (Abstraction & Reviews)	Q5.D15, Q5.D16, Q5.D17	Pressman §13.4; Mall §6.7	Procedural/data/control abstraction, stepwise refinement, Parnas information hiding, formal design review roles & checklist.